IntelliBug: An AI-Assisted System for Bug Severity Classification and Solution Recommendation for Web Applications

Nurfarhana Najihah Binti Mohd Fadli[1]
[1]Department of Computing, Universiti Teknologi PETRONAS, 32610 Bandar Seri Iskandar, Perak, Malaysia
Corresponding email: Nurfarhana_22003122@utp.edu.my[1]

Abstract
Software teams manually assign the severity of a bug report. This can cause prolonged delays and can be highly subjective and inconsistent. In this paper we present IntelliBug, a web application that can predict the severity of bug reports as either Critical, Medium or Low and recommend what the solution to the defect should be. The model was trained using an XGBoost classifier, using the titles of 478,653 Eclipse Bugzilla reports. The model used 10,000 TF-IDF features and was trained with balanced weights due to around 89% of the data being medium. The model is wrapped in a Flask application with Google sign-in functionality, MySQL database, dashboard and analytics. A large language model provides root cause, recommended solution and best practice for each report. The model had an accuracy of 55.5%, and a macro F1 score of 0.365. It was able to identify 51.9% of Critical reports and 49.7% of Low reports. The baseline of just predicting majority class found no reports. Finally, 12 Selenium tests were passed on the web application. Five users were able to complete all four unassisted tasks and achieved an average SUS score of 69.0. Five scorers gave recommendations scores between 4.14 and 4.34 out of five across the three criteria. This demonstrates that it is feasible to predict the severity of a text-based scenario to assist with triage, however the precision of the minority class is low. Next steps will be to train using full descriptions and scoring recommendations with practitioners.

Keywords XGBoost, TF-IDF, large language model, defect management, software testing, text mining

INTRODUCTION

Bug tracking systems store thousands of defect reports, and each report carries a severity level which guides the order of repair [1]. The level is usually entered by the reporter and reviewed by a triager, so the decision is subjective and often inconsistent [2]. Delays in the repair of severe defects affect users and release schedules.

Earlier studies predicted severity from the text of the report [3], [4], [5]. Most return a label only, and most are evaluated offline. The tester receives no repair guidance and no tool to follow the report until it is closed. Class imbalance adds a further problem, because a model which always predicts the dominant class reaches a high accuracy and finds no rare defect [6].

This paper introduces IntelliBug, a web application that can: 1) classify the severity of bugs using XGBoost; 2) suggest a solution to a bug using large language models; 3) provide a dashboard for defect tracking; and 4) be evaluated using machine learning metrics, automated functional testing, and a usability survey.

LITERATURE REVIEW

Menzies and Marcus applied text mining and rule learning to NASA defect reports and showed severity to be predictable from the words of a report [3]. Lamkanfi et al. used Naive Bayes on the short descriptions of Eclipse, Mozilla, and GNOME reports to separate severe from non-severe bugs [2], and a later comparison of four algorithms reported the best results for Naive Bayes Multinomial [4]. Tian et al. predicted fine-grained severity levels with an information-retrieval based nearest neighbour method [5]. Anvik et al. recommended the developer who should fix a bug [7]. Recent work has used Deep Learning models. CNN and BiLSTM models achieved 89%-91% accuracy on the Eclipse and Mozilla reports [8]. Fine-tuned BERT and GPT-2 models were used to extract sentiments from the reporter [9] [10]. A lightweight transformer was used with domain alignment to achieve a macro F1-score of 0.56 across the five levels of Mozilla reports [11].

Gradient Boosting uses many weak trees to create a highly accurate model [12]. XGBoost includes regularization and histogram-based training that considers sparse data [13]. Large language models can be used to provide human-readable repair advice. However, this advice needs to be verified. The reviewed studies stop at a label, so a tool which joins prediction, recommendation, and tracking, and evaluates the tool and not only the model, is missing.

METHODOLOGY

Dataset and Preprocessing

The Eclipse Bugzilla export of July 2022 was used [14]. Enhancement requests, invalid severity values, missing text, and duplicate identifiers were removed, which left 478,883 records. The six severity values were grouped into Critical (blocker, critical), Medium (major, normal) and Low (minor, trivial). All letters in the title were changed to lowercase. Codes, inside braces were removed. The text was then broken into tokens. Stop words in English were removed from the text. After 230 empty records were dropped, 478,653 records remained.

Features and Model

A TF-IDF vectoriser with 10,000 terms converted each title to a vector [15], [16]. The records were split 80% and 20% with stratification and a fixed seed, giving 382,922 training and 95,731 test records. Balanced sample weights, given in Equation (1), were applied during training [6], where n is the number of training samples, K is the number of classes and n_c is the number of samples of class c.

$$w_c = \frac{n}{K n_c} \tag{1}$$

The XGBoost classifier [13] used three hundred trees, a maximum depth of seven, a learning rate of 0.08, row and column subsampling of 0.8 and histogram-based trees. Performance was measured with accuracy and the macro-average F1-score in Equation (2), where $F1_c$ is the F1-score of class c.

$$\text{Macro} - \text{F1} = \frac{1}{K}\sum_{c=1}^{K} F1_c \tag{2}$$

System Design

IntelliBug was built with Flask [17], as shown in Figure 1. Users sign in with Google, submit a report with title, product, environment, description, and screenshot, and receive predicted severity. The title, description and severity are sent to the Claude model through its API [18], which returns a JSON object with a root cause, a suggested fix, and a best practice. The result is stored in MySQL and shown as a three-step diagram. The signed-in user filters every query, so each tester sees only the personal reports.

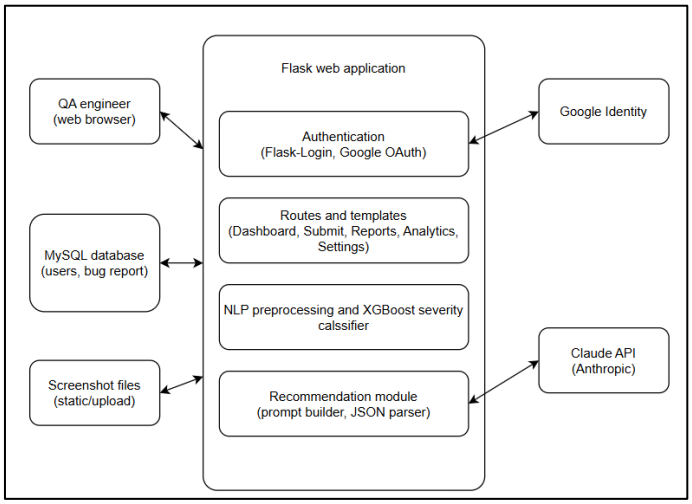


Figure 1: System architecture of IntelliBug

Evaluation Design

Twelve functional test cases were automated using Selenium WebDriver. Pytest [19]. Usability was evaluated using the System Usability Scale (SUS) [20]. Five participants completed four tasks. Answered ten questionnaire items. The average score was compared against the benchmark of sixty-eight [21]. Five raters assessed the recommendations in ten bug reports. The reports were categorized as three one Medium and six Low. Each recommendation was scored from one to five on relevance, correctness and actionability.

RESULTS

Table 1 gives test-set results. The majority-class baseline reached higher accuracy but found no Critical or Low report. The confusion matrix is shown in Figure 2.

Table 1: Test-set performance of the deployed model and a baseline

Model	Accuracy	Macro F1	Recall Critical	Recall Low
Always Medium	88.9%	31.4%	0%	0%
XGBoost (deployed)	55.5%	36.5%	51.9%	49.7%

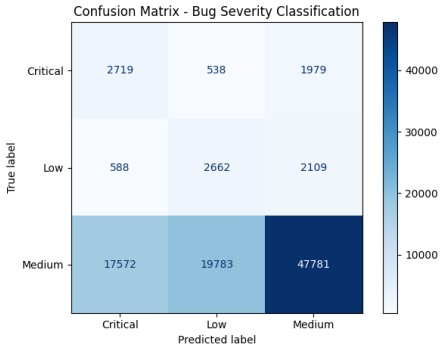


Figure 2: Confusion matrix of the deployed model on 95,731 test reports

Per-class precision was 13.0% for Critical, 11.6% for Low and 92.1% for Medium, because many medium reports were predicted as Critical (17,572) or Low (19,783). All twelve Selenium tests passed. Testing also exposed three defects, namely a form which accepted empty fields, a stale status after the Back button and reports of one user visible to another user, and all three were repaired and retested. Five participants completed all four tasks alone (100%), and the mean SUS score was 69.0, one point above the benchmark of sixty-eight. Three participants scored at or above the benchmark. Participants rated the learning effort and the integration of functions lowest. The raters gave a mean of 4.20 (standard deviation 0.78) for relevance, 4.34 (0.66) for correctness and 4.14 (0.78) for actionability over fifty ratings each, and no rater scored below 3.

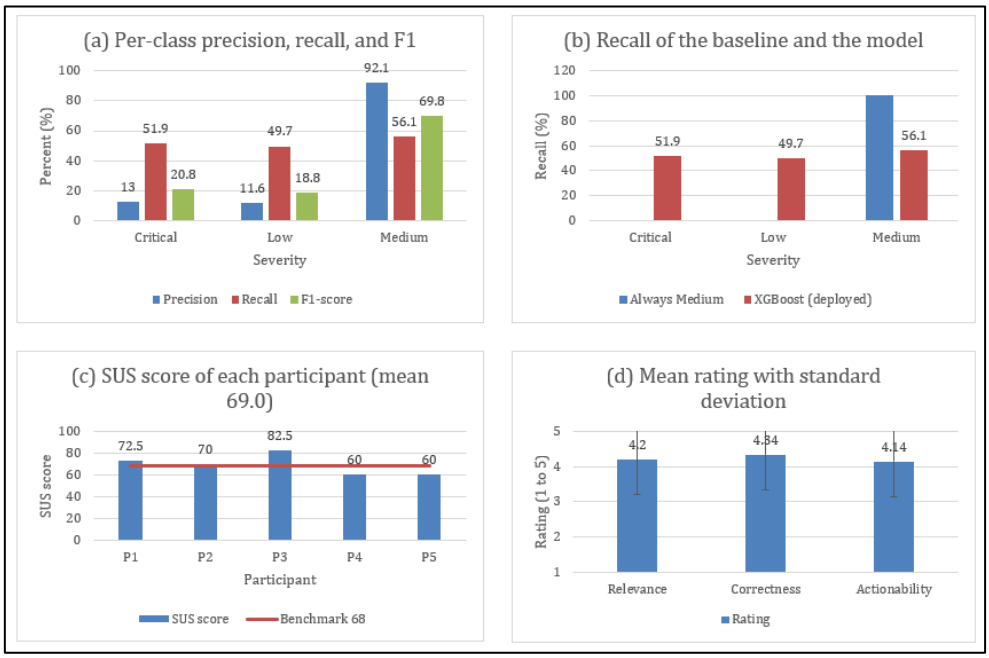


Figure 3: Analysis of the results. (a) Precision, recall, and F1-score per class. (b) Recall of the baseline and the deployed model. (c) SUS score of each participant. (d) Mean rating of the recommendations with standard deviation

Figure 3 shows an analysis of the results. (a) High precision was achieved for the Medium class (92.1%) and low precision for the other two classes (Critical 13.0%, Low 11.6%). All three classes achieved similar recall rates (~50%). (b) This shows how the model compares to a baseline. The baseline achieves perfect recall for medium reports (~100%) but misses any Critical and Low reports. The deployed model

loses 43.9 points of Medium recall but successfully achieves 51.9% of Critical and 49.7% of Low reports. (c) Each individual's SUS score is shown. Three people scored 70+ and two scored 60, below the benchmark score of 68. Finally, (d) provides averages ranging between 4.14 and 4.34. Correctness had the highest meaning and least variation. Relevance and actionability were more variable.

DISCUSSION

Balanced weights moved the model from ignoring the minority classes to finding about half of them, at the price of low precision. The system is therefore a triage aid whose output a tester confirms. Likely causes of the modest scores are the short title text, the noise of reporter-assigned severity [2], the merging of six levels into three and the overlap of vocabulary between classes. The model was trained on titles while users enter a title and a description. The usability example only includes five participants in the sample; therefore, it may not be representative of other products. The suggestions were made without looking at the applications source code. The people who gave the ratings were five friends of the writer. No expert looked at the advice. Compared it to the code. So the high scores show a sign of good quality, not a confirmed result.

CONCLUSIONS

IntelliBug is a web-based tool that can predict severity, recommend solutions and track defects. The model achieved a macro F1 score of 0.365. The model identified half of the Critical and Low reports. The automated tests were passed. The usability survey achieved a mean SUS score of 69.0. Future work will include training using full descriptions, comparing transformer models using cross validation, adding a confidence score and rating the recommendations by professional testers.

ACKNOWLEDGMENT

The authors thank the Department of Computing, Universiti Teknologi PETRONAS, for the guidance and facilities of the Final Year Project, and the participants of the usability test.

REFERENCES

[1] Mozilla, "The Bugzilla guide: Bug fields." [Online]. Available: https://bugzilla.readthedocs.io/ (accessed Oct. 2026).

[2] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *Proc. 7th IEEE Working Conf. Mining Software Repositories (MSR)*, Cape Town, South Africa, 2010, pp. 1–10.

[3] T. Menzies and A. Marcus, "Automated severity assessment of software defect reports," in *Proc. IEEE Int. Conf. Software Maintenance (ICSM)*, Beijing, China, 2008, pp. 346–355.

[4] A. Lamkanfi, S. Demeyer, Q. D. Soetens, and T. Verdonck, "Comparing mining algorithms for predicting the severity of a reported bug," in *Proc. 15th European Conf. Software Maintenance and Reengineering (CSMR)*, Oldenburg, Germany, 2011, pp. 249–258.

[5] Y. Tian, D. Lo, and C. Sun, "Information retrieval based nearest neighbor classification for fine-grained bug severity prediction," in *Proc. 19th Working Conf. Reverse Engineering (WCRE)*, Kingston, ON, Canada, 2012, pp. 215–224.

[6] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[7] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in *Proc. 28th Int. Conf. Software Engineering (ICSE)*, Shanghai, China, 2006, pp. 361–370.

[8] T. S. Mian and A. Alsaeedi, "Software bug severity prediction using convolutional neural network and BiLSTM models," in *Advances in Intelligent Computing Techniques and Applications (IRICT 2023)*, F. Saeed, F. Mohammed, and Y. Fazea, Eds. Cham, Switzerland: Springer, 2024, pp. 1–12.

[9] A. Ali, Y. Xia, Q. Umer, and M. Osman, "BERT based severity prediction of bug reports for the maintenance of mobile applications," *J. Syst. Softw.*, vol. 208, Art. no. 111898, Feb. 2024, doi: 10.1016/j.jss.2023.111898.

[10] M. A. Arshad, A. Riaz, R. Fatima, and A. Yasin, "SevPredict: Exploring the potential of large language models in software maintenance," *AI*, vol. 5, no. 4, pp. 2739–2760, Dec. 2024, doi: 10.3390/ai5040132.

[11] L. Zhu, S. Wiangsamut, and J. Polpinij, "Integrating lightweight transformers for cross-project bug severity classification: An applied AI approach in software engineering," *Appl. Sci.*, vol. 16, no. 12, Art. no. 6026, Jun. 2026, doi: 10.3390/app16126026.

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[14] A. Lamkanfi, J. Pérez, and S. Demeyer, "The Eclipse and Mozilla defect tracking dataset: A genuine dataset for mining bug information," in *Proc. 10th Working Conf. Mining Software Repositories (MSR)*, San Francisco, CA, USA, 2013, pp. 203–206.

[15] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[17] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/ (accessed Oct. 2026).

[18] Anthropic, "Claude API documentation." [Online]. Available: https://docs.claude.com (accessed Oct. 2026).

[19] Selenium Project, "Selenium documentation." [Online]. Available: https://www.selenium.dev/documentation/ (accessed Oct. 2026).

[20] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[21] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.